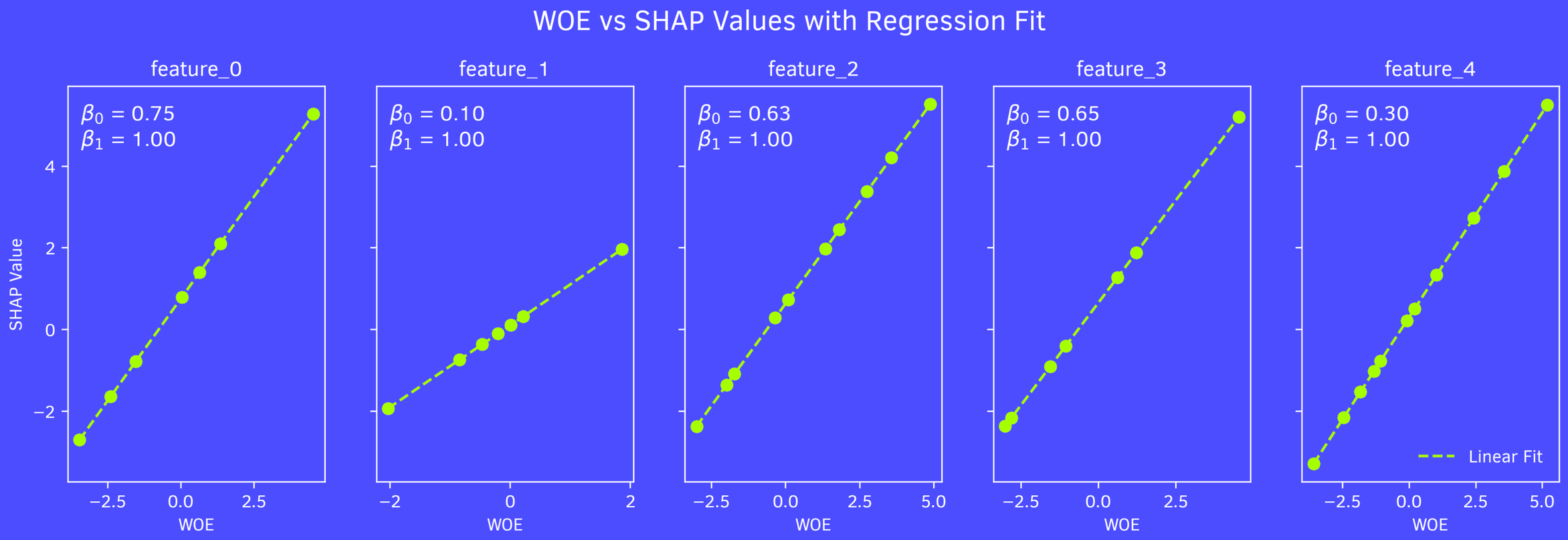
SHAP scoring

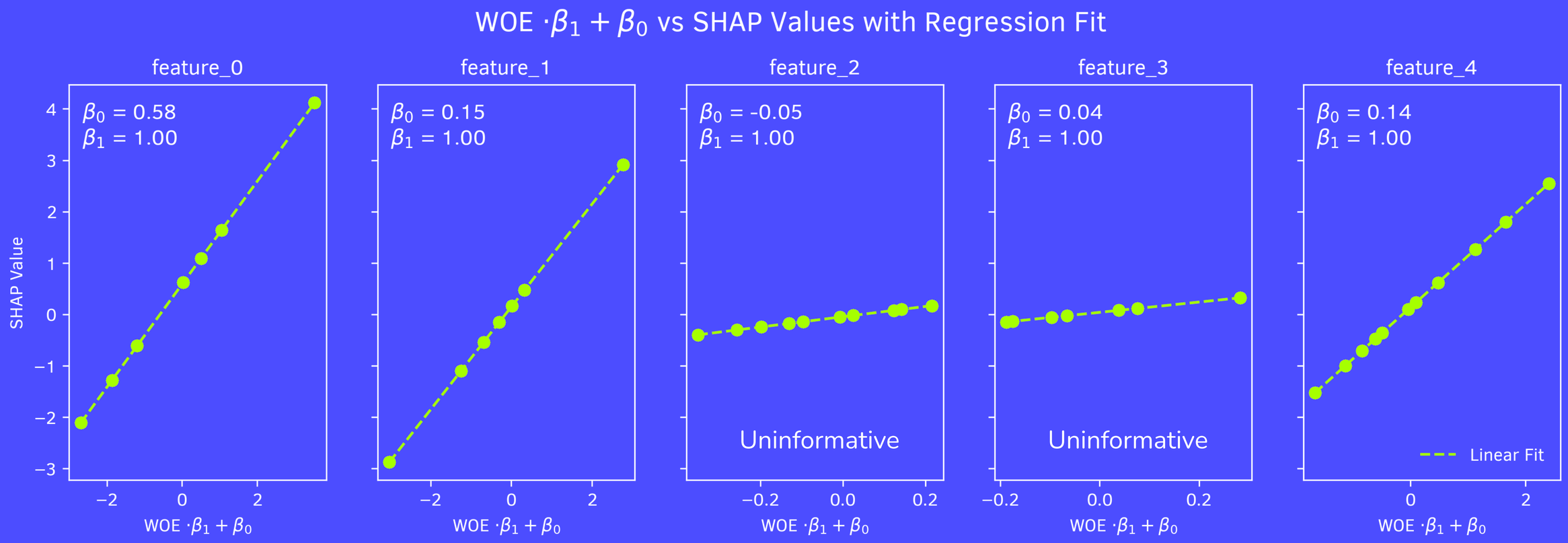
Relationship between WOE and linear SHAP values



Our starting point is to compare how Weight of Evidence (WOE) relates to SHAP values. The relationship between WOE and SHAP values is linear, and the intercept difference corresponds to the centering used by LinearExplainer in calculating the SHAP values ($\phi = \beta \times (X - \bar{X})$).

SHAP scoring

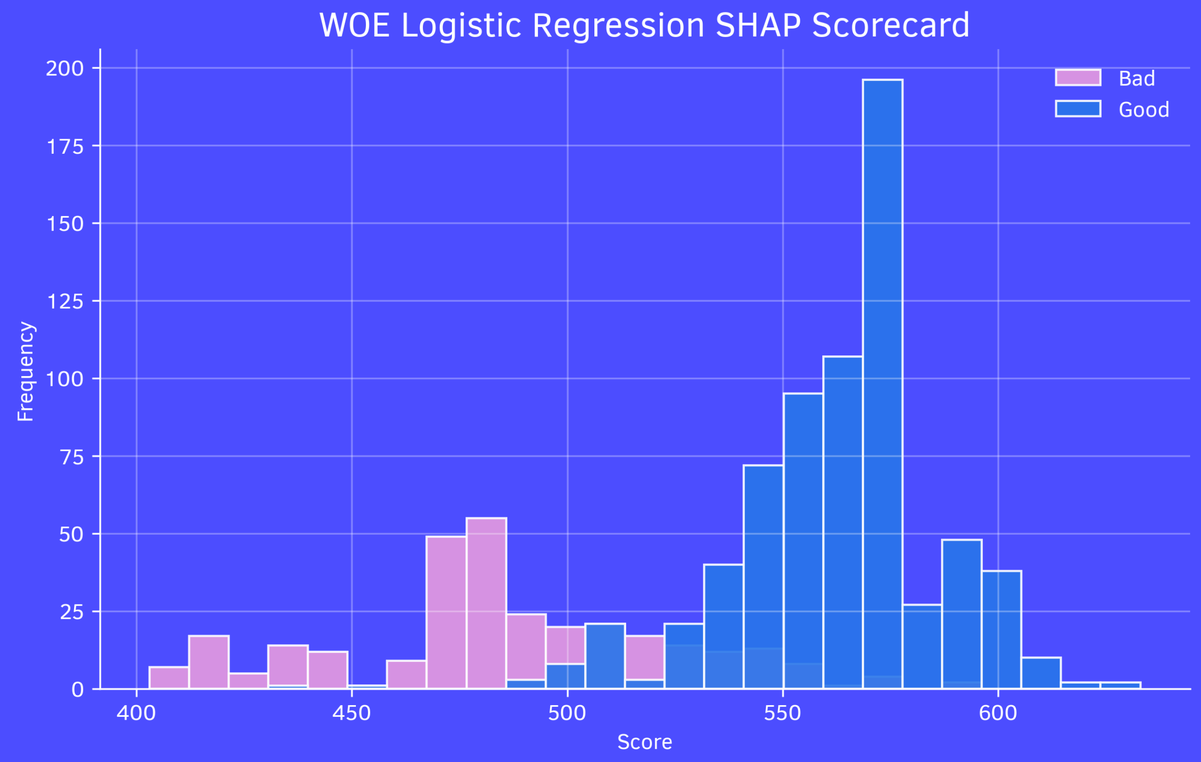
Relationship between WOE logistic regression and linear SHAP values

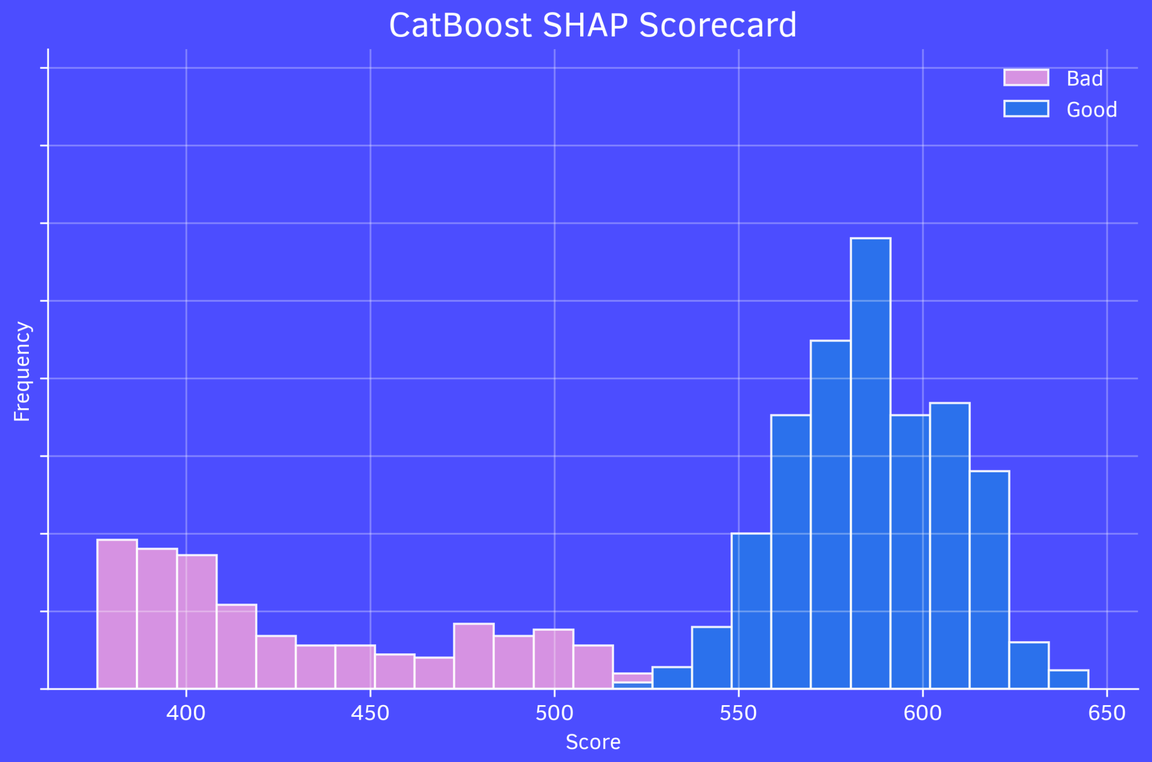


If we fit logistic regression model on top of WOE, the relationship between the model and SHAP still remains linear, but features 2 and 3 lose importance due to the joint optimization of parameters, which reduces the impact of uninformative features.

SHAP scoring

Scorecards based on linear and tree SHAP values

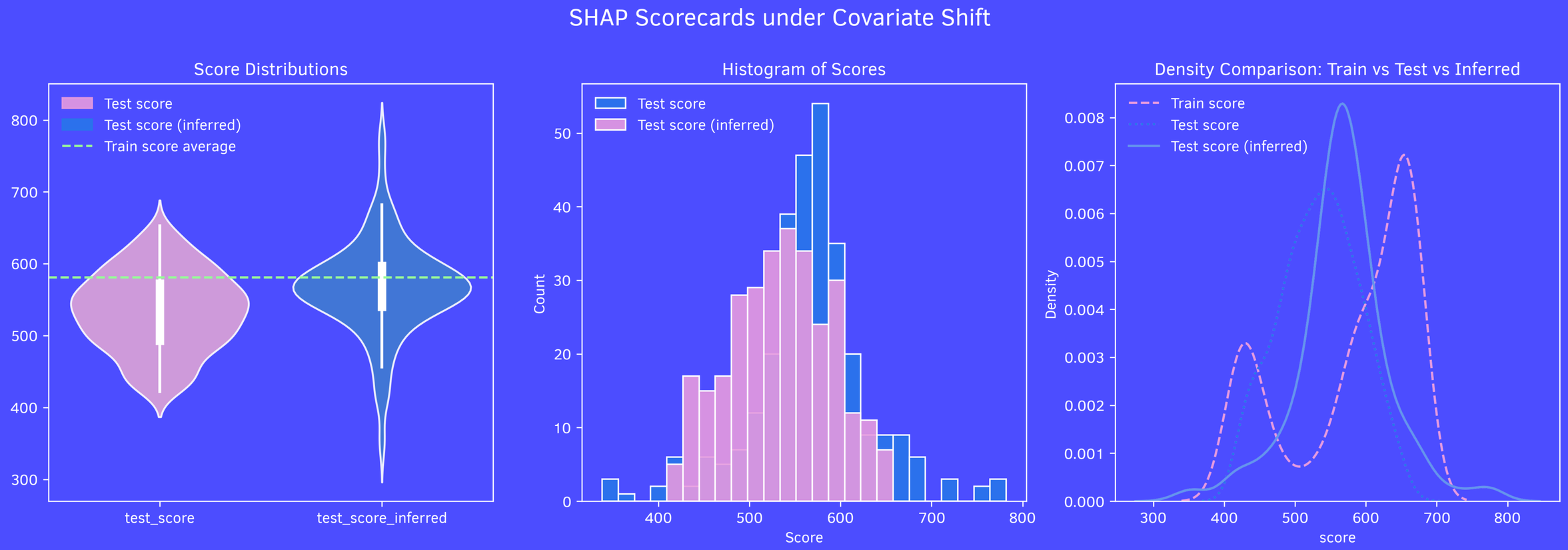




Since SHAP values for classification models are typically in log odds, we can leverage the additive property of SHAP values and create a scorecard using the PDO method (or any other method of logit scaling) for both linear and tree-based models.

SHAP scoring

Scorecards based on linear and tree SHAP values



SHAP values depend on the data distribution, and when test data shifts SHAP explanations change. By training a model to predict SHAP-based model score from raw features, we can infer scores on shifted data, potentially improving score stability and comparability under shift.

SHAP scoring

Scorecard code example

```python
import pandas as pd
import numpy as np
import shap
from scipy.special import logit

def compute_shap_scorecard(
   model, X, shap_df, model_type="tree",
   scorecard_dict={"pdo": 20, "target_points": 500, "target_odds": 19}
):
"""Convert SHAP values into a scorecard-style system."""

factor = scorecard_dict["pdo"] / np.log(2)
offset = scorecard_dict["target_points"] + factor * np.log(scorecard_dict["target_odds"])
if model_type == "tree":
   explainer = shap.TreeExplainer(model)
elif model_type == "linear":
   explainer = shap.LinearExplainer(model, X)
else:
   raise ValueError("Invalid model type: Choose 'tree' or 'linear'.")
shap_values = explainer.shap_values(X)
shap_df = pd.DataFrame(shap_values, columns=X.columns)
base_score = logit(np.mean(model.predict_proba(X)[:, 1]))
scorecard_df = pd.DataFrame()
for feature in X.columns:
   scorecard_df[f"{feature}_score"] = factor * -shap_df[feature] + base_score
scorecard_df["score"] = scorecard_df.sum(axis=1) + offset

return scorecard_df.round(0)
```

📒 Check out the full experiment here